AI Executive Assistant - API Documentation

Overview

The AI Executive Assistant API provides programmatic access to AI-powered content generation for AuthiChain. This API uses Anthropic's Claude Sonnet 4 for high-quality, context-aware content creation.

Base URL: `https://your-domain.com/api/ai-assistant`
Authentication: Required (session-based)
Rate Limit: 50 requests/hour per user
Model: Claude Sonnet 4 (claude-sonnet-4-20250514)

Authentication

All API requests require authentication. Users must be logged in with a valid session.

```
// Example: Using fetch with credentials
const response = await fetch('/api/ai-assistant', {
  method: 'POST',
  credentials: 'include', // Include session cookies
  headers: {
    'Content-Type': 'application/json',
  },
  body: JSON.stringify({
    action: 'draft_sales_email',
    data: { /* ... */ }
  })
});
```

Request Format

POST /api/ai-assistant

Headers:

```
Content-Type: application/json
Cookie: next-auth.session-token=...
```

Body:

```
{
  "action": "string (required)",
  "data": {
    // Action-specific fields
  }
}
```

Response Format

Success Response (200)

```
{
  "success": true,
  "result": "Generated content string",
  "metadata": {
    "category": "sales",
    "action": "draft_sales_email",
    "timestamp": "2025-10-20T10:30:00.000Z",
    "user": "user@example.com"
  }
}
```

Error Responses

401 Unauthorized

```
{
  "error": "Unauthorized - Please sign in"
}
```

400 Bad Request

```
{
  "error": "Missing required field: name"
}
```

429 Rate Limit Exceeded

```
{
  "error": "Rate limit exceeded. Please try again in an hour."
}
```

500 Internal Server Error

```
{
  "error": "AI Assistant failed to generate content"
}
```

Available Actions

GET /api/ai-assistant

Returns list of all available actions and their requirements.

Response:

```
{
  "message": "AuthiChain AI Executive Assistant API",
  "version": "1.0.0",
  "actions": {
    "sales": [ /* ... */ ],
    "marketing": [ /* ... */ ],
    // ... other categories
  },
  "usage": {
    "endpoint": "/api/ai-assistant",
    "method": "POST",
    "body": {
      "action": "string (required)",
      "data": "object (required fields vary by action)"
    }
  }
}
```

Actions Reference

Sales & Outreach

draft_sales_email

Generate personalized sales email.

Action: `draft_sales_email`

Required Fields:
- `name` (string): Prospect name
- `company` (string): Company name
- `title` (string): Job title
- `industry` (string): Industry

Optional Fields:
- `painPoints` (array): Pain points to address

Example Request:

```
{
  "action": "draft_sales_email",
  "data": {
    "name": "Sarah Chen",
    "company": "Luxury Brand Co.",
    "title": "Chief Digital Officer",
    "industry": "Fashion",
    "painPoints": ["counterfeits", "brand protection"]
  }
}
```

Example Response:

```
{
  "success": true,
  "result": "Subject: Transform Your Brand Protection...\n\nHi Sarah,\n\n...",
  "metadata": {
    "category": "sales",
    "audience": "Luxury Brand Co."
  }
}
```

---

draft_partnership_email

Generate partnership proposal email.

Action: `draft_partnership_email`

Required Fields:
- `name` (string): Contact name
- `company` (string): Company name
- `type` (string): Partnership type
- `proposedIntegration` (string): Integration description

Optional Fields:
- `benefits` (array): Mutual benefits

Example Request:

```
{
  "action": "draft_partnership_email",
  "data": {
    "name": "Michael Zhang",
    "company": "BlockMarket",
    "type": "Marketplace Integration",
    "proposedIntegration": "Two-way NFT verification API",
    "benefits": ["Expanded user base", "Enhanced credibility"]
  }
}
```

Marketing Content

generate_linkedin_post

Create LinkedIn post.

Action: `generate_linkedin_post`

Required Fields:
- `topic` (string): Post topic

Optional Fields:
- `options.includeStats` (boolean): Include statistics
- `options.tone` (string): professional | conversational | thought-leader
- `options.targetAudience` (string): Specific audience

Example Request:

```
{
  "action": "generate_linkedin_post",
  "data": {
    "topic": "NFT authentication in luxury brands",
    "options": {
      "tone": "professional",
      "targetAudience": "luxury brand executives"
    }
  }
}
```

---

generate_blog_post

Write blog post.

Action: `generate_blog_post`

Required Fields:
- `topic` (string): Blog topic

Optional Fields:
- `length` (string): short | medium | long (default: medium)
- `keywords` (array): Target keywords for SEO

Example Request:

```
{
  "action": "generate_blog_post",
  "data": {
    "topic": "How blockchain prevents counterfeits",
    "length": "medium",
    "keywords": ["NFT", "authentication", "blockchain", "anti-counterfeit"]
  }
}
```

---

generate_social_content

Generate platform-specific social media content.

Action: `generate_social_content`

Required Fields:
- `platform` (string): twitter | linkedin | instagram | facebook
- `topic` (string): Content topic

Optional Fields:
- `style` (string): professional | casual | educational

Example Request:

```
{
  "action": "generate_social_content",
  "data": {
    "platform": "twitter",
    "topic": "New NFT collection launch",
    "style": "professional"
  }
}
```

---

generate_email_campaign

Create email campaign.

Action: `generate_email_campaign`

Required Fields:
- `subject` (string): Campaign subject
- `audience` (string): Target audience
- `goal` (string): Campaign goal

Optional Fields:
- `keyPoints` (array): Key points to cover
- `tone` (string): Email tone

Example Request:

```
{
  "action": "generate_email_campaign",
  "data": {
    "subject": "Introducing AuthiChain Pro",
    "audience": "Free tier users",
    "goal": "Upgrade to Pro subscription",
    "keyPoints": ["Unlimited NFTs", "Advanced features", "Priority support"],
    "tone": "professional"
  }
}
```

NFT-Specific Content

generate_nft_description

Generate NFT description.

Action: `generate_nft_description`

Required Fields:
- `name` (string): NFT name
- `category` (string): NFT category
- `creator` (string): Creator name

Optional Fields:
- `story` (string): Background story
- `rarity` (string): Rarity level

- `attributes` (object): NFT attributes
- `edition` (string): Edition info

Example Request:

```
{
  "action": "generate_nft_description",
  "data": {
    "name": "Cyber Phoenix #42",
    "category": "Digital Art",
    "creator": "ArtistDAO",
    "story": "A mythical phoenix reborn in the digital realm",
    "rarity": "Ultra Rare",
    "attributes": {
      "background": "Neon Cityscape",
      "style": "Cyberpunk"
    }
  }
}
```

---

generate_collection_description

Generate collection description.

Action: `generate_collection_description`

Required Fields:
- `name` (string): Collection name
- `theme` (string): Collection theme
- `totalItems` (number): Total items
- `creator` (string): Creator name

Optional Fields:
- `utilities` (array): Collection utilities
- `roadmap` (string): Roadmap info
- `communityBenefits` (string): Community benefits

Example Request:

```
{
  "action": "generate_collection_description",
  "data": {
    "name": "Verified Luxury Collection",
    "theme": "High-end fashion authentication",
    "totalItems": 500,
    "creator": "Luxury Brand Collective",
    "utilities": ["Exclusive events", "Early access", "Discounts"]
  }
}
```

---

generate_nft_marketing

Generate NFT marketing copy.

Action: `generate_nft_marketing`

Required Fields:
- `name` (string): NFT name
- `category` (string): NFT category

Optional Fields:
- `price` (string): Price
- `highlights` (array): Key highlights
- `targetBuyers` (string): Target buyers
- `purpose` (string): listing | social | email | auction

Example Request:

```
{
  "action": "generate_nft_marketing",
  "data": {
    "name": "Golden Era #1",
    "category": "Collectible",
    "price": "2.5 ETH",
    "purpose": "auction",
    "highlights": ["First edition", "Artist signed", "Rare attributes"]
  }
}
```

Customer Support

generate_support_response

Generate support response.

Action: `generate_support_response`

Required Fields:
- `category` (string): Issue category
- `question` (string): User question

Optional Fields:
- `userContext` (object): User context (accountType, platform, etc.)
- `urgency` (string): low | normal | high

Example Request:

```
{
  "action": "generate_support_response",
  "data": {
    "category": "NFT Minting",
    "question": "How do I mint my first NFT?",
    "userContext": {
      "accountType": "basic",
      "platform": "mobile"
    },
    "urgency": "normal"
  }
}
```

generate_onboarding_email

Create onboarding email.

Action: `generate_onboarding_email`

Required Fields:
- `name` (string): User name
- `accountType` (string): basic | pro | enterprise
- `step` (string): welcome | first_mint | explore_features | upgrade_prompt

Optional Fields:
- `signupDate` (string): Signup date
- `progress` (object): User progress data

Example Request:

```
{
  "action": "generate_onboarding_email",
  "data": {
    "name": "Alex",
    "accountType": "pro",
    "step": "first_mint",
    "signupDate": "2025-10-15"
  }
}
```

---

generate_faq_answer

Generate FAQ answer.

Action: `generate_faq_answer`

Required Fields:
- `question` (string): FAQ question

Optional Fields:
- `category` (string): Question category

Example Request:

```
{
  "action": "generate_faq_answer",
  "data": {
    "question": "What are gas fees and how do I pay them?",
    "category": "NFT Minting"
  }
}
```

Executive & Analytics

generate_daily_briefing

Generate executive briefing.

Action: `generate_daily_briefing`

Optional Fields (all):
- `metrics.nftsMinted` (number): NFTs minted
- `metrics.revenue` (number): Revenue
- `metrics.activeAuctions` (number): Active auctions
- `metrics.newSignups` (number): New signups
- `metrics.enterpriseLeads` (number): Enterprise leads
- `metrics.topCollection` (string): Top collection
- `metrics.avgMintTime` (string): Average mint time

Example Request:

```
{
  "action": "generate_daily_briefing",
  "data": {
    "metrics": {
      "nftsMinted": 127,
      "revenue": 3450,
      "activeAuctions": 23,
      "newSignups": 45,
      "enterpriseLeads": 3
    }
  }
}
```

---

analyze_competitor

Analyze competitor.

Action: `analyze_competitor`

Required Fields:
- `name` (string): Competitor name
- `features` (array): Competitor features

Optional Fields:
- `pricing` (string): Pricing info
- `strengths` (array): Strengths
- `weaknesses` (array): Weaknesses
- `marketPosition` (string): Market position

Example Request:

```
{
  "action": "analyze_competitor",
  "data": {
    "name": "CompetitorNFT",
    "features": ["Basic minting", "Marketplace", "Collections"],
    "pricing": "$49/month",
    "strengths": ["Large user base", "Brand recognition"],
    "weaknesses": ["Limited blockchain support", "No enterprise features"]
  }
}
```

Utility Tools

summarize_text

Summarize text.

Action: `summarize_text`

Required Fields:
- `text` (string): Text to summarize

Optional Fields:
- `maxWords` (number): Max words (default: 100)
- `style` (string): concise | detailed

Example Request:

```
{
  "action": "summarize_text",
  "data": {
    "text": "Long article text here...",
    "maxWords": 150,
    "style": "concise"
  }
}
```

---

improve_writing

Improve writing.

Action: `improve_writing`

Required Fields:
- `text` (string): Text to improve

Optional Fields:
- `style` (string): professional | casual | technical
- `purpose` (string): general | marketing | technical

Example Request:

```
{
  "action": "improve_writing",
  "data": {
    "text": "Our nft platform is really good and helps people.",
    "style": "professional",
    "purpose": "marketing"
  }
}
```

Code Examples

JavaScript/TypeScript (Frontend)

```
// Function to call AI Assistant API
async function generateContent(action: string, data: any) {
  try {
    const response = await fetch('/api/ai-assistant', {
      method: 'POST',
      credentials: 'include',
      headers: {
        'Content-Type': 'application/json',
      },
      body: JSON.stringify({ action, data })
    });

    if (!response.ok) {
      const error = await response.json();
      throw new Error(error.error);
    }

    const result = await response.json();
    return result.result;

  } catch (error) {
    console.error('AI Assistant Error:', error);
    throw error;
  }
}

// Example usage
const salesEmail = await generateContent('draft_sales_email', {
  name: 'John Doe',
  company: 'Tech Corp',
  title: 'CTO',
  industry: 'Technology'
});

console.log(salesEmail);
```

Node.js (Backend)

```
const AIExecutiveAssistant = require('./lib/AIExecutiveAssistant');

async function generateSalesEmail() {
  const assistant = new AIExecutiveAssistant(process.env.ANTHROPIC_API_KEY);

  const email = await assistant.draftSalesEmail({
    name: 'Sarah Chen',
    company: 'Luxury Brand Co.',
    title: 'Chief Digital Officer',
    industry: 'Fashion',
    painPoints: ['counterfeits', 'brand protection']
  });

  console.log(email);
}

generateSalesEmail();
```

React Component

```jsx
import { useState } from 'react';

function AIContentGenerator() {
  const [result, setResult] = useState('');
  const [loading, setLoading] = useState(false);

  const generateLinkedInPost = async () => {
    setLoading(true);

    try {
      const response = await fetch('/api/ai-assistant', {
        method: 'POST',
        credentials: 'include',
        headers: { 'Content-Type': 'application/json' },
        body: JSON.stringify({
          action: 'generate_linkedin_post',
          data: {
            topic: 'NFT authentication trends',
            options: { tone: 'professional' }
          }
        })
      });

      const data = await response.json();
      setResult(data.result);

    } catch (error) {
      console.error('Error:', error);
    } finally {
      setLoading(false);
    }
  };

  return (
    <div>
      <button onClick={generateLinkedInPost} disabled={loading}>
        {loading ? 'Generating...' : 'Generate Post'}
      </button>
      {result && <pre>{result}</pre>}
    </div>
  );
}
```

Rate Limiting

Default Limits

- **50 requests per hour** per user
- Rate limit resets every hour
- In-memory storage (consider Redis for production scaling)

Headers (Future)

Future versions may include rate limit headers:

```
X-RateLimit-Limit: 50
X-RateLimit-Remaining: 45
X-RateLimit-Reset: 1697811600
```

Handling Rate Limits

```javascript
async function generateWithRetry(action, data, maxRetries = 3) {
  for (let i = 0; i < maxRetries; i++) {
    try {
      const response = await fetch('/api/ai-assistant', {
        method: 'POST',
        credentials: 'include',
        headers: { 'Content-Type': 'application/json' },
        body: JSON.stringify({ action, data })
      });

      if (response.status === 429) {
        // Rate limited - wait and retry
        const waitTime = Math.pow(2, i) * 1000; // Exponential backoff
        await new Promise(resolve => setTimeout(resolve, waitTime));
        continue;
      }

      return await response.json();

    } catch (error) {
      if (i === maxRetries - 1) throw error;
    }
  }
}
```

Error Handling

Common Errors

Status Code	Error	Solution
400	Missing required field	Check API docs for required fields
401	Unauthorized	User must be logged in
403	Forbidden	Admin access required (if enabled)
429	Rate limit exceeded	Wait for rate limit to reset
500	Internal server error	Check logs, contact support

Error Response Example

```
{
  "error": "Missing required field: name",
  "status": 400,
  "timestamp": "2025-10-20T10:30:00.000Z"
}
```

Best Practices

1. Cache Results

```
// Cache generated content to avoid repeated API calls
const cache = new Map();

async function getCachedContent(action, data) {
  const key = `${action}-${JSON.stringify(data)}`;

  if (cache.has(key)) {
    return cache.get(key);
  }

  const result = await generateContent(action, data);
  cache.set(key, result);

  return result;
}
```

2. Handle Errors Gracefully

```
try {
  const result = await generateContent('draft_sales_email', data);
  // Success
} catch (error) {
  if (error.message.includes('Rate limit')) {
    showToast('Rate limit exceeded. Please try again later.');
  } else if (error.message.includes('Unauthorized')) {
    redirectToLogin();
  } else {
    showToast('Failed to generate content. Please try again.');
    logError(error);
  }
}
```

3. Validate Input

```
function validateSalesEmailData(data) {
  const required = ['name', 'company', 'title', 'industry'];

  for (const field of required) {
    if (!data[field]) {
      throw new Error(`Missing required field: ${field}`);
    }
  }

  return true;
}
```

4. Provide User Feedback

```
// Show loading state
setLoading(true);

// Generate content
const result = await generateContent(action, data);

// Update UI
setResult(result);
setLoading(false);
showSuccessMessage('Content generated successfully!');
```

Testing

Unit Tests

```
const { generateContent } = require('./ai-assistant-client');

describe('AI Assistant API', () => {
  it('should generate sales email', async () => {
    const result = await generateContent('draft_sales_email', {
      name: 'Test User',
      company: 'Test Co',
      title: 'CEO',
      industry: 'Tech'
    });

    expect(result).toContain('Test User');
    expect(result).toContain('Test Co');
  });

  it('should handle missing fields', async () => {
    await expect(
      generateContent('draft_sales_email', { name: 'Test' })
    ).rejects.toThrow('Missing required field');
  });

  it('should handle rate limits', async () => {
    // Make 51 requests to exceed limit
    for (let i = 0; i < 51; i++) {
      try {
        await generateContent('draft_sales_email', { /* ... */ });
      } catch (error) {
        expect(error.message).toContain('Rate limit exceeded');
      }
    }
  });
});
```

Changelog

Version 1.0.0 (October 20, 2025)

- Initial release
- 18 actions across 6 categories
- Rate limiting (50 req/hour)
- Session-based authentication
- Full TypeScript support

Support

API Issues: support@authichain.com
Documentation: https://docs.authichain.com/ai-assistant
Status: https://status.authichain.com

Last Updated: October 20, 2025
API Version: 1.0.0